

DEVHIRE

Plateforme de recrutement et de gestion des stages / PFE

Objectif

Construire un projet complet, presentable sur GitHub et sur un CV, permettant de mettre en pratique les principales notions du parcours Spring de Navin Reddy - en excluant volontairement Spring AI.

Type	Application web full-stack
Architecture cible	Microservices + API Gateway + communication asynchrone
Backend	Java / Spring Boot
Frontend	Angular
Base de donnees	PostgreSQL
Infrastructure	Docker / Docker Compose
Projet portfolio	Concu pour demontrer les competences utiles avant un stage PFE

Principe directeur: construire le projet progressivement, version par version, en reutilisant les notions apprises dans le cours au moment ou elles deviennent pertinentes.

1. Vue d'ensemble

DevHireest une plateforme derecrutement et de stages qui met en relation des candidats et des recruteurs. Elle permet de publier des offres, candidater, gerer les dossiers, planifier des entretiens et notifier les utilisateurs.

Le projet vise a depasser le simple CRUD: il doit montrer securite, architecture, communication entre services, traitement asynchrone, conteneurisation et une vraie interface web.

2. Utilisateurs et roles

- CANDIDATE - consulte les offres, filtre/recherche, sauvegarde des offres, postule, suit ses candidatures, gere son profil et son CV, consulte ses entretiens et notifications.
- RECRUITER - cree et gere des offres, consulte les candidatures, fait evoluer les statuts, organise les entretiens et envoie des decisions.
- ADMIN - administre la plateforme, supervise les comptes et peut moderer les donnees selon les besoins du projet.

3. Fonctionnalites principales

Authentification et securite

- Inscription et connexion.
- JWT pour l'authentification des API.
- Spring Security pour proteger les ressources.
- Autorisation basee sur les roles.
- Gestion des erreurs d'authentification et d'accès.

Gestion des offres

- Creation, modification et suppression d'offres par les recruteurs.
- Type d'offre: PFE, stage, emploi, etc.
- Competences requises.
- Localisation, mode remote/presentiel/hybride.
- Salaire ou gratification si pertinent.
- Recherche, filtres et pagination cote candidat.
- Sauvegarde d'offres par le candidat.

Gestion des candidatures

Flux metier cible: CANDIDATE -> APPLY -> RECRUITER REVIEW -> INTERVIEW -> ACCEPTED / REJECTED.

- Statuts: PENDING, REVIEWING, INTERVIEW, ACCEPTED, REJECTED.
- Consultation des candidatures cote candidat et recruteur.
- Historique des changements de statut recommande.
- Validation des entrees et gestion propre des erreurs.

Gestion des CV

- Upload d'un CV PDF.
- Stockage des metadonnees du fichier.
- Association du CV au profil candidat.
- Possibilite d'evoluer plus tard vers un stockage objet tel que S3 si necessaire.

Notifications et entretiens

- Notification lors d'une décision de candidature.
- Notifications internes à la plateforme.
- Email optionnel.
- Création d'entretiens: date, heure, type et lien de réunion.
- Consultation des entretiens par le candidat et le recruteur.

4. Architecture cible

L'architecture finale separe les domaines fonctionnels en services Spring Boot autonomes. L'API Gateway constitue le point d'entree principal.

Client Web				
Auth Service Users / JWT / roles	Job Service Offres / skills	Application Service Candidatures	Interview Service Entretiens	Notification Service Notifications
PostgreSQL	PostgreSQL	PostgreSQL	PostgreSQL	PostgreSQL

Communication asynchrone

Une file de messages (par exemple RabbitMQ ou Kafka, selon le contenu du cours et le choix technique retenu) permet de decoupler les actions importantes, notamment la creation de notifications a la suite d'un changement de statut de candidature.

Exemple: Application Service publie un evenement ApplicationStatusChanged -> Notification Service le consomme -> creation d'une notification et, si active, envoi d'un email.

5. Services du systeme

	Description des services
Auth Service	Comptes, login, JWT, roles, securite.
Job Service	Offres, competences requises, recherche/filtres.
Application Service	Candidatures, statuts, historique et logique de candidature.
Interview Service	Planification et gestion des entretiens.
Notification Service	Notifications internes et integration email.
API Gateway	Point d'entree, routage, centralisation des appels.

6. Stack technique

	Technologie
Langage	Java
Backend	Spring Boot
Securite	Spring Security + JWT
Acces donnees	Spring Data JPA / Hibernate
API	REST
Build	Maven
Base de donnees	PostgreSQL
Microservices	Services Spring Boot independants

Gateway	Spring Cloud Gateway
Service discovery / config	A ajouter selon les sections du cours et le besoin du projet
Messagerie	RabbitMQ ou Kafka
Conteneurisation	Docker + Docker Compose
Frontend	Angular
Versioning	Git + GitHub
Documentation API	Swagger / OpenAPI

7. Strategie de developpement par versions

V1 - Monolithe

- Spring Boot + PostgreSQL
- Users, offres et candidatures
- REST API
- Structure propre du projet et couche service/repository

V2 - Securite

- SpringSecurity
- JWT
- Roles CANDIDATE / RECRUITER / ADMIN
- Protection des routes et autorisations

V3 - Backend avance

- JPA/Hibernateapprofondi
- Pagination
- Filtrage et recherche
- Validation
- Gestion centralisee des exceptions
- Swagger/OpenAPI

V4 - Passage en microservices

- SeparerAuth,Jobs,Applications,Notifications et Interviews
- Communication HTTP entre services
- Contrats d'API clairement definis

V5 - Infrastructure

- APIGateway
- Config Server / Service Discovery selon le contenu retenu du cours
- Docker
- Docker Compose

V6 - Communication asynchrone

- RabbitMQou Kafka
- Evenements metier
- Decouplage du Notification Service
- Traitement asynchrone

V7 - Frontend

- Angular
- Authentification
- Dashboards distincts
- Recherche d'offres
- Gestion des candidatures et entretiens
- Integration desAPIs

8. Interface frontend cible

Le frontend doit donner une vraie experience utilisateur plutot qu'un simple ecran CRUD. Les dashboards changent selon le role.

- Candidate Dashboard: offres recommandees, candidatures en cours, prochains entretiens, notifications, profil et CV.
- Recruter Dashboard: offres actives, nombre de candidatures, candidats a examiner, entretiens planifies.
- Navigation: barre laterale ou menu clair, pages protegees selon le role, recherche et filtres utilisables.
- UX: formulaires valides, messages d'erreur comprehensibles, etats de chargement, pagination et feedback apres chaque action importante.

9. Dockerisation

L'objectif final est de pouvoir lancer l'environnement avec une seule commande.

`docker compose up`

Le compose doit regrouper les services applicatifs, les bases PostgreSQL necessaires, le broker de messages et le frontend. La structure peut evoluer au fil du projet.

10. Exemple de flux metier complet

- Etape 1: un candidat cree son compte et se connecte. Auth Service genere un JWT.
- Etape 2: il consulte les offres via l'API Gateway. Job Service fournit les offres et filtres.
- Etape 3: il postule. Application Service cree la candidature et son statut initial PENDING.
- Etape 4: le recruteur consulte la candidature et la passe a REVIEWING.
- Etape 5: le recruteur planifie un entretien. Interview Service cree le rendez-vous.
- Etape 6: l'etat de candidature devient INTERVIEW. Un evenement est publie sur le broker.
- Etape 7: Notification Service consomme l'evenement et envoie la notification correspondante.
- Etape 8: apres l'entretien, la candidature passe a ACCEPTED ou REJECTED et le cycle est termine.

11. Organisation recommandee du repository GitHub

Une organisation claire facilite la lecture par un recruteur ou un encadrant.

```
devhire/  
├── frontend/  
├── api-gateway/  
├── auth-service/  
├── job-service/  
├── application-service/  
├── interview-service/  
├── notification-service/  
├── docker-compose.yml  
├── README.md  
└── docs/
```

- README avec objectif, architecture, stack, instructions de lancement et captures d'ecran.
- Diagramme d'architecture et description des services.
- Instructions Docker et variables d'environnement.
- Collection Postman ou documentation Swagger si utile.
- Historique Git propre avec commits comprehensibles.

12. Ce que le projet doit demontrer sur un CV

- Developpement backend avec Java et Spring Boot.
- Conception et consommation d'APIs REST.
- Securisation d'une application avec Spring Security et JWT.
- Acces aux donnees avec PostgreSQL, JPA et Hibernate.
- Architecture microservices et communication inter-services.
- Communication asynchrone avec un broker de messages.
- Conteneurisation avec Docker et orchestration locale avec Docker Compose.
- Developpement frontend Angular.
- Utilisation professionnelle de Git et GitHub.

13. Formulation CV proposee

DevHire - Recruitment & Internship Management Platform
Java, Spring Boot, Spring Security, JWT, Angular PostgreSQL,
JPA/Hibernate, Microservices, Docker, RabbitMQ, REST APIs

Developed a full-stack recruitment and internship management platform using a microservices architecture, implementing JWT-based authentication, role-based access control, REST APIs, asynchronous communication, PostgreSQL persistence, Docker containerization, and dedicated

dashboards for candidates and recruiters.

14. Regle de qualite pour le projet

Le projet doit rester un projet personnel compris de bout en bout. Chaque technologie presente dans le README et sur le CV doit correspondre a quelque chose que tu as reellement implemente, teste et que tu peux expliquer en entretien.

Le but n'est pas d'accumuler des technologies. Le but est de montrer une progression coherente: fonctionnalites -> securite -> donnees -> architecture -> communication -> Docker -> frontend.